Østfold University College

Assignment 2:
Numeric model training with Histogram of Oriented Gradients & Principle Component Analysis

ITF31719-1 26V Bildeanalyse

Andreas Isaksen

April 15, 2026
Halden

Contents

Appendices

Introduction

1.1 Description

In this document I will be describing the theory, implementation and solution for the two main techniques used in this assignment: Histogram of Oriented Gradients (HOG) and Principal Component Analysis (PCA). These techniques are fundamental in the field of computer vision and machine learning, and they play a crucial role in feature extraction and dimensionality reduction, respectively. The goal of this assignment is to apply these techniques to a dataset of images and evaluate their performance in accordance with the requirements of the assignment.

1.2 Goal

A minimum of 3 different models must be trained, tested and evaluated on the received dataset, and the results must be presented in a clear and concise manner. The models will be trained using the HOG features extracted from the images, and PCA will be applied to reduce the dimensionality of the feature space before training the models.

The performance of the models will be evaluated using appropriate metrics, and the results will be discussed in relation to the theory and implementation of HOG and PCA.

HOG must be handwritten and conseptually explained, while **PCA** can be implemented using a library such as scikit-learn, but the theory and implementation must still be explained to the best of my ability.

If feasible within the time frame of the assignment, I will also explore the possibility of storing the trained models and the extracted features, and further try to do tests on new data that was not part of the original dataset, to evaluate the generalization capabilities of the models and my own personal abilities in training and testing machine learning models.

1.3 Structure

The solution has been structured as a python project, with the main training script located in `./train/app/train_models.py`. The HOG implementation can be found in `./my_packages/hog.py`, and the PCA implementation can be found in `./my_packages/utils.py`.

The overall structure of the project is organized in a way that allows for easy navigation and understanding of the different components of the solution, while also adhering to best practices in software development. The code is well-documented and follows a clear and consistent style, making it easy to read and understand.

1.3.1 Project tree structure

```
root
+---artifacts
¦   +---logs
¦   +---models
¦   +---outputs
¦
+---data
¦   +---Digit_testing
¦   +---Digit_training
¦
+---my_package
¦       data_loader.py
¦       hog.py
¦       inference.py
¦       logging_utils.py
¦       models.py
¦       preprocessing.py
¦       training.py
¦       utils.py
¦       __init__.py
¦
+---test
¦   +---app
¦   ¦       real_image_eval.py
¦   +---img
¦
+---train
    +---app
            train_models.py
```

- **artifacts**
  Intended for storing logs, trained models and output files such as evaluation results and plots.

- **data**
  Directory for storing the dataset. In this case, it contains two subdirectories: `Digit_testing` and `Digit_training`, which hold the testing and training images, respectively.

- **my_package**
  Directory for storing custom Python packages and utility functions. It contains various modules for data loading, HOG implementation, inference, logging utilities, model definitions, preprocessing, training, and other utility functions.

- **test**
  Directory for storing test scripts and evaluation code. It contains two subdirectories:

  1. `app` which contains the script `real_image_eval.py` which is an experiment to evaluate the trained models on new, real-world images that were not part of the original dataset.

2. `img` which is intended for storing images used in testing and evaluation.

- **train**
  Directory for storing training scripts. It contains the main training script `train_models.py`, which is responsible for orchestrating:

  - loading the dataset,
  - extracting HOG features,
  - applying PCA for dimensionality reduction,
  - training multiple machine learning models, and
  - evaluating their performance.
  - storing the trained models and evaluation results in the `artifacts` directory for later analysis and comparison.
  - returns a final terminal output with the evaluation results of the trained models.

Theory, Implementation and Solution

2.1 Histogram of Oriented Gradients

2.1.1 Theory

"**HOG** is a feature descriptor used in computer vision and image processing for object detection. It captures the structure or the shape of an object by analyzing the distribution (histograms) of gradient orientations in localized portions of an image. HOG is especially popular for human and vehicle detection."[1].

This technique works by dividing the image into small connected regions called cells, and for each cell, it computes a histogram of gradient directions or edge orientations for the pixels within the cell. The combined histograms from all cells form the HOG descriptor, which can be used as input for machine learning algorithms to perform object detection.
Though HOG is effective for capturing local shape information, it can be sensitive to changes in lighting and pose, and may not perform well in cluttered backgrounds or with small objects.

2.1.2 Implementation

1. **Preprocessing**
   Though it's not directly mentioned in the course slide covering HOG, It's common to perform some preprocessing steps on the input image before computing the HOG features. This can include resizing the image to a standard size, converting it to grayscale, and applying normalization techniques to enhance contrast and reduce noise.

   Since the dataset used in this assignment consists of grayscale images with a resolution of 28x28 pixels, we can skip the resizing and grayscale conversion steps. However, we can still apply normalization to improve the contrast and reduce noise in the images.

2. **Gradient Calculation**
   The next step is to compute the magnitude and direction of the gradient for each pixel in the image. This is typically done using simple filters such as the Sobel operator, which calculates the gradient in both the horizontal (x) and vertical (y) directions.

   For this assignment, I've reused the implementation of the Sobel operator from Assignment 1, which computes the gradients for each pixel in the image.[A]

3. **Build Cell Histograms**
   After calculating the gradients, the image is divided into small spatial regions called cells (in this case 4×4 pixels). For each cell, a histogram of gradient orientations is constructed using a fixed number of bins (typically 9 bins covering the range $0°$ to $180°$). Each pixel within the cell contributes to the histogram based on its gradient magnitude and orientation, where stronger gradients have a larger influence.

   In this implementation, the histogram is computed by assigning each pixel's orientation to the corresponding bin and accumulating its magnitude. This results in a compact representation of the local edge structure within each cell.

4. **Compute Cell Histogram**
   For each individual cell, the gradient magnitude and orientation values are flattened into one-dimensional arrays to simplify iteration. The orientation of each pixel is mapped to a histogram bin using predefined bin edges, and the corresponding gradient magnitude is added to that bin. Special care is taken to handle boundary cases, such as orientations equal to $180°$, ensuring they are assigned to the final bin.

   The resulting histogram encodes the distribution of edge directions within the cell, forming the fundamental building block of the HOG feature descriptor.

5. **Block Normalization**
   To improve the robustness of the HOG features to changes in illumination and contrast, the cell histograms are normalized over larger spatial regions called blocks (in this case 2×2 cells). The blocks are extracted using a sliding window approach, resulting in overlapping regions across the image.

   For each block, the histograms of the contained cells are concatenated into a single dimensional vector, which is then normalized using the L2-norm. A small constant ϵ is added to the norm to avoid division by zero.

   This normalization step reduces the influence of local intensity variations and enhances the stability of the feature representation.

2.1.3 Solution

The full solution for this can be found in the code project in the file `./my_packages/hog.py`, which is imported and used in the main training script `./train/app/train_models.py` with the function `extract_hog_dataset()`.

2.2 Principle Component Analysis

2.2.1 Theory

"PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components."[2]

With the usage of linear algebra, we can transform the original data into a new coordinate system, where the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on.

PCA is widely used in data analysis and machine learning for tasks such as data visualization, noise reduction, and feature extraction. It can help to identify patterns in data and express the data in a way that highlights their similarities and differences. However, PCA assumes that the principal components are linear combinations of the original features, which may not always capture complex relationships in the data. Additionally, PCA is sensitive to the scale of the features, so it's important to standardize the data before applying PCA.

2.2.2 Implementation

PCA has been implemented in this assigment with a function with the following structure:

Step1: Get original feature dimension.
This is done by checking the shape of the input data, which is typically a 2D array where rows represent samples and columns represent features. The number of columns gives us the original feature dimension.

Step2: Decide reduced dimension. The reduced dimension is a hyperparameter that we can choose based on the desired level of dimensionality reduction. It can be determined by analyzing the explained variance ratio or by setting a specific number of principal components to retain.

Step3: Create PCA object. We can create a PCA object using a library such as scikit-learn, which provides a convenient implementation of PCA. Principal components are computed based on the original features, and the PCA object will store the necessary information for transforming the data.

Step4: Fit PCA on training data. The PCA object is fitted to the training data, which involves computing the covariance matrix of the data, calculating the eigenvalues and eigenvectors, and selecting the top principal components based on the explained variance.

Step5: Transform training and test data. Once the PCA model is fitted, we can use it to transform both the training and test data by projecting them onto the selected principal components. This results in a new representation of the data with reduced dimensionality while retaining as much variance as possible.

2.2.3 Solution

The full solution for this can be found in the code project in the file `./my_packages/utils.py`, which is imported and used in the main training script `./train/app/train_models.py` with the function `apply_pca()`.

Part 3 — Discussion

3.1 Training

The task of training the models was accomplished without any major issues, and the training process was relatively smooth. The HOG features were successfully extracted from the images, and PCA was applied to reduce the dimensionality of the feature space before training the models.

Initially three different *keras* models were trained and evaluated on the dataset, and the results were promising with all models scoring 95% or above in accuracy. The models were able to learn from the HOG features and make accurate predictions on the test set. Further down I decided to train a fourth model, which was a simple logistic regression model implemented using scikit-learn, to compare the performance of a more traditional machine learning model with the deep learning models. The logistic regression model also performed well, indicating that the HOG features allready are linearly separable to a good degree, and that the deep learning models may not have been necessary for this particular dataset.

Performance of the models was evaluated using sklearn's `accuracy_score` which gets stored under `artifacts/outputs/` as a .txt file, and the results were discussed in relation to the theory and implementation of HOG and PCA.

Overall, the training process was considered successful, and the models were able to learn from the HOG.features and make accurate predictions on the test set. The results were consistent with the expectations based on the theory and implementation of HOG and PCA, and the models demonstrated good performance on the dataset.

3.2 Testing

This part of the of the delivery goes a little further than what was scoped in the assignment, but I wanted to test the trained models on new, real-world images that were not part of the original dataset, to evaluate the generalization capabilities of the models and my own personal abilities in training and testing machine learning models.

The testing process involved collecting a small set of real-world images of handwritten and printed digits, which were then preprocessed and fed into the trained models for evaluation. The results were rather underwhelming, with most values being incorrectly classified while whitespace and background noise were confidently classified as digits.

Overall, the testing process revealed that while the models performed well on the original dataset, they struggled to generalize to new, real-world images. This highlights the importance of evaluating machine learning models on diverse and representative datasets to ensure their robustness and generalization capabilities in real-world applications.

After several atempt to improve the performance of the models on the new images and long conversations with LLMs, I've ended up with the following possible improvements that could be made to increase the performance of the models on new, real-world images:

- The model needs a feature representation of background noise and whitespace, so that it can learn to distinguish between digits and non-digit elements in the images.
- Different mask sizes and shapes could be experimented with in the HOG feature extraction process to capture more relevant features from the images.

Bibliography

[1] *Histogram of Oriented Gradients*. GeeksforGeeks. 17:32:00+00:00. URL: https://www.geeksforgeeks.org/computer-vision/histogram-of-oriented-gradients/ (visited on 04/14/2026).

[2] *Principal Component Analysis (PCA)*. GeeksforGeeks. 18:43:59+00:00. URL: https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/ (visited on 04/14/2026).

[3] OpenAI. *Project Structure Suggestions*. ChatGPT. URL: https://chatgpt.com/share/69dd54e9-d5d4-8397-95a4-51128923fbd4.

[4] OpenAI. *Save Model PCA Check*. ChatGPT. URL: https://chatgpt.com/share/69debc7a-27d8-8388-a7e7-b48c32526eb2.

[5] OpenAI. *ModuleNotFoundError Fix*. ChatGPT. URL: https://chatgpt.com/share/69debcb7-e378-8386-aa7b-82402a009931.

[6] OpenAI. *Project Conclusion Feedback*. ChatGPT. URL: https://chatgpt.com/share/69debbc5-e7f4-8386-ba41-46019024fd1c.

[7] OpenAI. *Sliding Window Approach*. ChatGPT. URL: https://chatgpt.com/share/69debced-84d8-8385-8ba6-4302cc0eec71.

Østfold University College

Assignment I:
Canny Edge Detection From Scratch

ITF31719-1 26V Bildeanalyse

Andreas Isaksen

March 9, 2026
Halden

Contents

Appendices

Part 1 – Implementation

1.1 Gaussian Smoothing

1.1.1 Generate a 2D Gaussian kernel manually

This requirement is solved in the function `Gsmooth(path, sigma: float )` in attached file *ob1lib.py*.

Requirements:

- "Kernel size must depend on sigma."
  This was solved by implementing the logic on how "a Gaussian kernel requires $6\sigma_t - 1$ values"[1]
  The kernel size is then defined by this calculation in the `Gsmooth` function with this logic:
  `size = int(6 * sigma - 1)`
- "Normalize the kernel so that the sum equals 1."
  This was solved by dividing each element in the created kernel with the sum of the kernel values. This way the summary of the elements would be 1 (or at least verry close) while still keeping the weighted calculations: `kernel /= np.sum(kernel)`.
- "Implement 2D convolution manually."
  Convolution is implemented with a nested `for` loop that iterates though each pixel in the image that fits the *kernel size* and performs the weighted calculations on each *pixel* which is the stored in a `out` array object of the same dimention as the imported image:

```
for i in range(k, H-k):
    for j in range(k, W-k):
        s = 0.0
        for a in range(kH):
            for b in range(kW):
                s += kernel[a, b] * img[i +(a-k), j + (b-k)]
        out[i, j] = s
return out
```

1.1.2 Show results for at least two sigma values

The images below shows the result of the *Gaussian filter* with $\sigma = 1 \wedge 3$. For comparison the original image has been included in the image row.

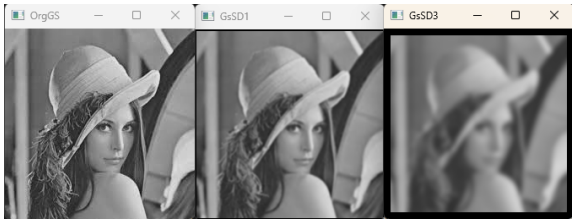

Figure 1.1: Screenshot of Original Grasyscale and Gaussian blur with $\sigma = 1 \wedge 3$

1.2 Gradient Computation

This requirement is solved in the function `GradComp(img)` in attached file *ob1lib.py*.

Requirements:

- "Manually define Sobel kernels."
  This kernel is based on the *Sobel kernels* for Y/X that's defined in Appendix A.

```
sobel_y = np.array([
    [-1,-2,-1],
    [0,0,0],
    [1,2,1]
], dtype=np.float32)
sobel_x = np.array([
    [-1,0,1],
    [-2,0,2],
    [-1,0,1]
], dtype=np.float32)
```

- "Perform convolution manually to compute Gx and Gy."
  This is solved by defining the range in the image (`region = img[y-1:y+2, x-1:x+2]`) to convolute and multiply with the *Y/X sobel kernels* for each iteration to find the approximated derivatives. This is then stored in two arrays lists with a corresponding shape to the input image.

```
for y in range(1, H-1):
    for x in range(1,W-1):
        region = img[y-1:y+2, x-1:x+2]
        gy[y, x] = np.sum(region * sobel_y)
        gx[y, x] = np.sum(region * sobel_x)
```


- "Compute gradient magnitude and direction using arctan2."
  1. With the approximated derivatives from *X/Y sobel operation*, the magnitude is found with the operation $|\nabla I| = \sqrt{G_x^2 + G_y^2}$.

```
magnitude = np.sqrt(gy**2 + gx**2)
```

  2. Orientation is found by element-wise finding the *arc tangent* from the *Y/X Sobel* arrays. `np.arctan2` works well for this as it returns the radians.
     The radians are then converted to to degrees with; $Deg = \frac{180}{\pi} \cdot Rad$.[2]
     Lastly the range is restricted to $[0, 180)$ with modulo to get the *direction*.

```
angle = (np.arctan2(gy, gx) * 180 / np.pi) % 180
```

- Display gradient magnitude image.
  The images below illustrates the result from *X/Y sobel operation* and the *Isotropic(Gradient decent)*.
  In this assignment I've chosen to operate with relative strength of edges. Because of this I've had to normalize the magnitude array to a $0, 255$ value scale for visualization:

```
isotropic = (magnitude / (magnitude.max() + 1e-8)) * 255
```

Figure 1.2: Screenshot of Gradient magnitude with *Sobel_Y*, *Sobel_X* and *Isotropic*.

1.3 Non-Maximum Suppression

This requirement is solved in the function `nms(magnitude, angle)` in attached file *ob1lib.py*.

- "Quantize gradient direction into 4 sectors (0°, 45°, 90°, 135°)."
  This was solved by dividing the gradient directions into sections with if statements:

```
# Check for Horizontal gradient (0°) -- Edge runs horizontal.
if (0 <= a < 22.5) or (157.5 <= a < 180):
```



```
# Check for Horizontal gradient (45°) -- Edge runs diagonal.
elif (22.5 <= a < 67.5):
# Check for Horizontal gradient (90°) -- Edge runs vertically.
elif (67.5 <= a < 112.5):
# Check for Horizontal gradient (135°) -- Edge runs diagonal.
elif (112.5 <= a < 157.5):
```

- "Compare each pixel with its two neighbors along the gradient direction."
  For each defined sector the given neighbors are defined according to a $3x3$ grid:

```
# Horizontal.
[y, x-1]
[y, x+1]
# diagonal.
[y+1, x-1]
[y-1, x+1]
# vertically.
[y-1, x]
[y+1, x]
# diagonal other direction.
[y-1, x-1]
[y+1, x+1]
```

- "Keep only local maxima; suppress others to zero."
  With the results from the last step, we're now able to do a value comparison of center pixel vs. sector neighbors. If either of the neighbors have a higher value than the center pixel, then the value is set to 0. Else the pixel value remains unchanged:

```
# Trim edge lines according to neighbour
# If both neighbours have a lower value: Keep value
if magnitude[y, x] >= q and magnitude[y, x] >= r:
    out[y, x] = magnitude[y, x]
# If either neighbour has a higher value: Set to 0
else:
    out[y, x] = 0
```

1.4 Double Threshold and Hysteresis

This requirement is solved in the function `DT(nms, high=0.08, low=0.01, strong=255, weak=75)` and `hysteresis(dt,strong=255, weak=75)` in attached file *ob1lib.py*.

- "Select appropriate high and low thresholds."
  For this solution the high/low threshold has be set with a percentile relative to the highest value in the *Non-Maximum Suppression* output, which means that good threshold are generally found on a rather low percentile ($> 10\%$).

- "Classify pixels as strong, weak, or non-edges."
  This is solved with a `for` loop with `if` statements that classify the values accordingly to threshold percentile.
  By default these are the followign classifications:
  - $(y, x) \geq high = 255$
  - $(y, x) \geq low = 75$
  - $(y, x) < low = 0$
- "Implement hysteresis using 8-neighborhood connectivity."
  Firstly a *matrix fo neighbour vectors* gets defined. This will be used to connect the center pixel to its *8 relative offset pixel*.

```
nbrs = np.array([(1, -1),  (1, 0),  (1, 1),
                 (0, -1),           (0, 1),
                 (-1, -1), (-1, 0), (-1, 1)])
```

  Then *hysteresis* will iterate through the *DT* output.

  - Strong pixels will remain unchanged.
  - Weak pixels will be evaluated respectively to its 8 neighbors.
    * If any neighbors holds a strong value, then the weak pixel gets promoted to strong.
    * If no strong neighbors are present, the weak gets suppressed.

```
if any(dt[y+dy, x+dx] == strong for dy, dx in nbrs):
    out[y, x] = strong
else:
    out[y, x] = 0
```

- "Display final Canny edge image."

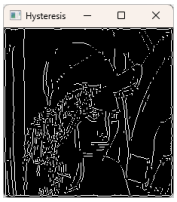

Figure 1.3: Screenshot of final image after textithysteresis

Part 2 – Experimental Analysis

The following images has been generated with the use of the `canny(path, sigma=1, high=0.08, low=0.01)` function, which combines all the function from chapter 1 into one single function with the following setup:

```
def canny(path, sigma=1, high=0.08, low=0.01):
    smooth = Gsmooth(path, sigma)
    mag,ang,_,_ = GradComp(smooth)
    nonmax = nms(mag,ang)
    dt,s,w = DT(nonmax, high, low)
    out = hysteresis(dt,s,w)
    return out
```

2.1 Test at least three different sigma values

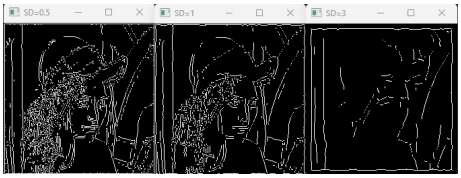

Figure 2.1: Canny Edge run with $\sigma = 0.5 \wedge 1 \wedge 3$

2.2 Test at least three different threshold pairs

Images below shows an extention of the images from 2.1. Here each σ value has been run with three different thresholds (threshold values are listed from left to right in accordance to the displayed images):

1. $High = max \cdot 0.1 \wedge Low = max \cdot 0.05$
2. $High = max \cdot 0.05 \wedge Low = max \cdot 0.01$
3. $High = max \cdot 0.03 \wedge Low = max \cdot 0.005$

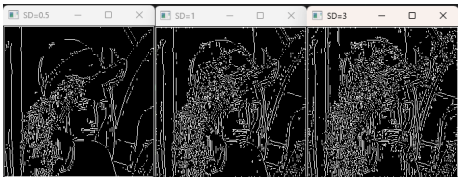

Figure 2.2: Canny edge run with $\sigma = 0.5$

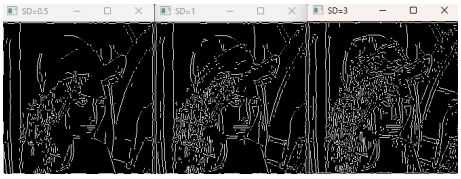

Figure 2.3: Canny edge run with $\sigma = 1$

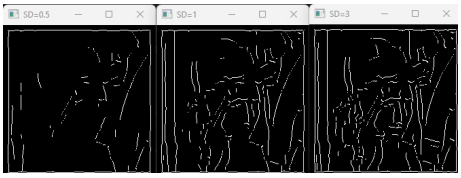

Figure 2.4: Canny edge run with $\sigma = 3$

2.3 Briefly discuss how parameters affect noise, edge thickness, and continuity

As seen in 2.1 and 2.2 there is a strong relation between *smoothing* and *Double thresholding* that must be balanced according to the intended usage.

Smoothing(σ)	A low σ value will allow **CED** to keep much noise, which can be good if more detailed edges are important for the analysis, but this might obscure the results as unwanted edges also might get included in *hysteresis*.
	A high σ value will remove much of the noise, which results in cleaner edges. But might also result in loss of fine details and reduce accuracy for edge localization in *hysteresis*.
High	Too high value might exclude weaker but valid edges.
	Too low value introduces noise to the result.
Low	Too high value might result in losing *edge continuity* on valid lines.
	Too low value might result in noise propagation.

Part 3 — Discussion

3.1 Why is Non-Maximum Suppression necessary?

NMS is a necessary step to "remove extra boxes that are detected around the same object"[3]. The role of a *NMS* process is to:

- **Reduce Redundancy:**
  "By removing overlapping boxes, NMS ensures that a single, high-confidence bounding box represents each object."[3]
- **Improve Precision:**
  "By retaining the bounding box with the highest confidence score, NMS enhances the accuracy of the detection results. It reduces number of false positives."[3]
- **Computational Efficiency:**
  "Improve the computational efficiency of the object detection process by reducing number of boxes to a manageable and relevant set."[3]

3.2 Why is hysteresis better than single thresholding?

Hysteresis uses two thresholds: a high threshold to detect strong edges and a low threshold to identify weaker edge pixels. Weak edges are only preserved if they are connected to strong edges through an *8-neighborhood*. This allows true edges with varying gradient strength to remain continuous while suppressing isolated noise responses.

With this *Hysteresis* produces a more continuous and reliable edge detection compared to single thresholding.

3.3 Why is Canny more robust than Sobel alone?

The trick lies in how **Canny** combines several processing steps that improves the noise *handling*, edge *localization* and edge *continuity*. The **Sobel** operator only computes the image gradient to highlight intensity changes, which makes it sensitive to noise and often produces thick or fragmented edges.

3.4 Identify limitations of your implementation.

- The current implementation does not apply *padding* during convolution in `Gsmooth`. As a result, pixels near the image borders are not processed in the same way as interior pixels, producing a zero-valued frame around the output image. This effect becomes more pronounced for larger σ values, since a larger Gaussian kernel increases the width of the unprocessed border region.
- The *hysteresis* stage only performs a single-pass neighbourhood check. This means that a weak edge pixel is only preserved if it is directly connected to a strong edge in that one iteration. In cases where edge continuity depends on chains of weak pixels, parts of a true edge may be suppressed. A recursive or iterative propagation strategy could improve connectivity.
- The double-threshold step is based on values relative to the maximum gradient magnitude in the current image. While this makes the method adaptive, it also makes the result sensitive

to image content. A few very strong gradients may dominate the threshold scaling, causing weaker but meaningful edges to be discarded.

- In the non-maximum suppression stage, gradient directions are quantized into only four principal angles ($0°$, $45°$, $90°$, and $135°$). This simplifies the implementation, but it is only an approximation of the true gradient direction. As a result, edge thinning may become less precise for edges whose directions fall between these sectors.
- The implementation relies on explicit nested loops for convolution, gradient computation, non-maximum suppression, thresholding, and hysteresis.This is computationally inefficient compared to vectorized NumPy operations or optimized library functions. The performance cost becomes more noticeable for large images.
- The Gaussian kernel size is defined as a function of σ, but the expression used (1.1.1) may produce even or very small kernel sizes for certain parameter values. Since convolution kernels are usually preferred to have odd dimensions with a well-defined center pixel, additional safeguards could make the implementation more robust.
- The current pipeline assumes grayscale processing only. Although this is standard for many edge-detection tasks, it means that edge information carried mainly by color differences is lost before processing begins.

Bibliography

[1] *Gaussian Filter*. In: *Wikipedia*. Nov. 17, 2025. URL: https://en.wikipedia.org/w/index.php?title=Gaussian_filter&oldid=1322606530 (visited on 02/26/2026).

[2] *Radians to Degrees - Conversion, Formula, Solved Examples & Calculator*. GeeksforGeeks. 0:00:01+00:00. URL: https://www.geeksforgeeks.org/maths/radians-to-degrees/ (visited on 03/06/2026).

[3] *What Is Non-Maximum Suppression?* GeeksforGeeks. 18:38:08+00:00. URL: https://www.geeksforgeeks.org/computer-vision/what-is-non-maximum-suppression/ (visited on 03/09/2026).

[4] John Canny. "A Computational Approach To Edge Detection." In: *Pattern Analysis and Machine Intelligence, IEEE Transactions on* PAMI-8 (Dec. 1, 1986), pp. 679–698. ISSN: 9780080515816. DOI: 10.1109/TPAMI.1986.4767851.

[5] *Apply a Gauss Filter to an Image with Python*. GeeksforGeeks. Apr. 28, 2026. URL: https://www.geeksforgeeks.org/python/apply-a-gauss-filter-to-an-image-with-python/ (visited on 02/26/2026).

[6] *NumPy Documentation — NumPy v2.4 Manual*. URL: https://numpy.org/doc/stable/index.html (visited on 02/26/2026).

[7] OpenAI. *imgOblig1Assist*. ChatGPT. URL: https://chatgpt.com/share/69aeacd2-55fc-800e-91b5-a08d0cdd8561.

[8] *Python Arithmetic Operators*. URL: https://www.w3schools.com/python/python_operators_arithmetic.asp (visited on 02/26/2026).

[9] *Python Assignment Operators*. URL: https://www.w3schools.com/python/python_operators_assign.asp (visited on 02/26/2026).

[10] *Python Try Except*. URL: https://www.w3schools.com/python/python_try_except.asp (visited on 02/26/2026).